CNN을 활용한 Fashion MNIST 분류

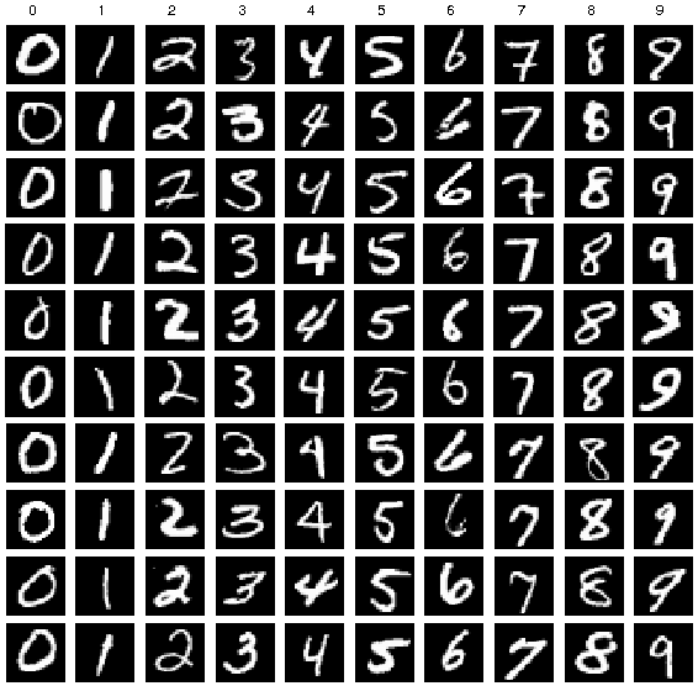
산업경영공학과 12210588 고우현

개요

- 주제 선정 이유
- 데이터 소개 및 전처리
- 모델 구성
- 모델 컴파일 및 학습
- 학습 결과 및 예측
- 결론

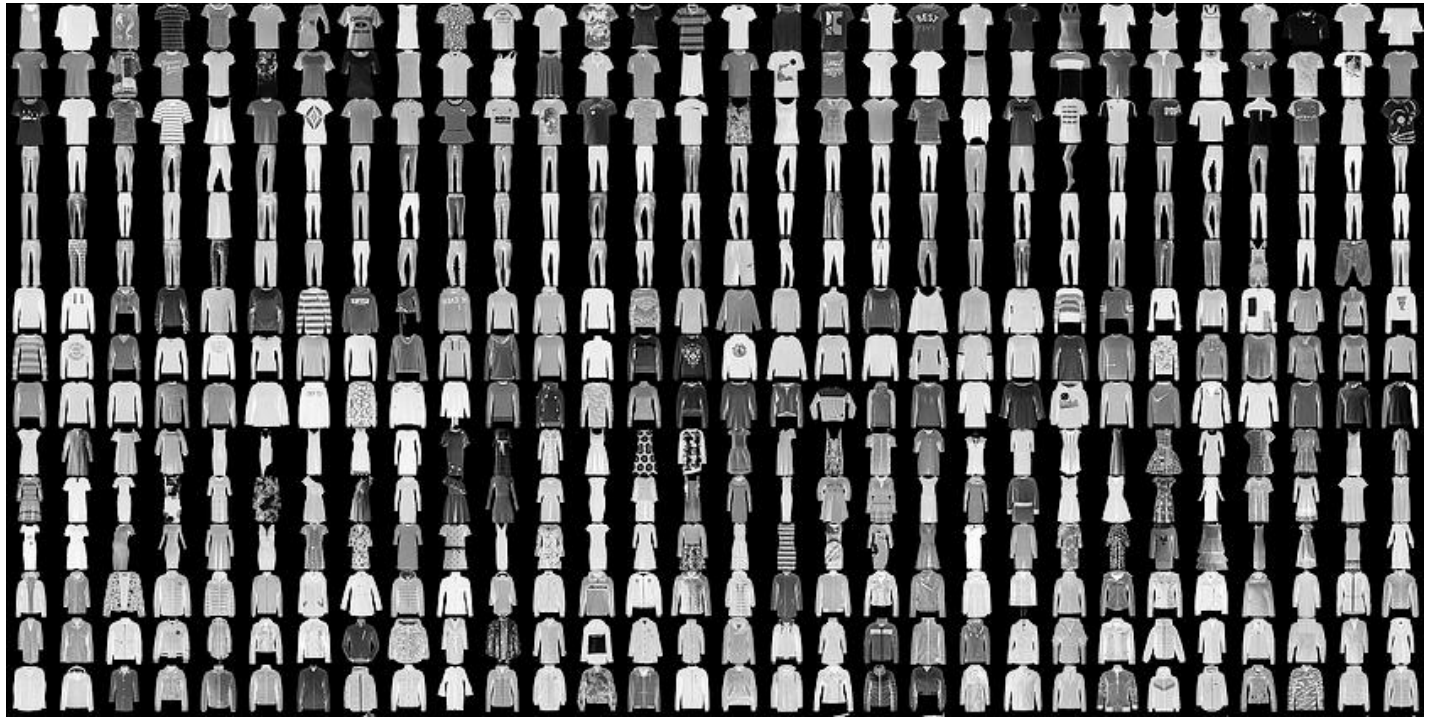
주제 선정 이유

- 딥러닝 강의 시간에 MNIST 손글씨를 예시로 활용해 딥러닝에 대해 배움
- 다른 MNIST 이미지를 분류는 어떻게 진행되는지 기말과제를 통해 이해함



데이터 소개 및 전처리

- 60000개의 train 이미지와 10000개의 test 이미지를 가짐
- 28 X 28의 사이즈를 지니며 총 10개의 클래스로 라벨과 함께 분류



https://www.kaggle.com/code/faressayah/fashion-mnist-classification-using-cnns/notebook

데이터 소개 및 전처리

- T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot 클래스로 분류
- 총 784개의 픽셀로 구성
- 각 픽셀 value는 0에서 255 사이의 정수로 구성

데이터 소개 및 전처리

In [1]:

```
# import libraries
import pandas as pd # Import Pandas for data manipulation using dataframes
import numpy as np # Import Numpy for data statistical analysis
import matplotlib.pyplot as plt # Import matplotlib for data visualisation
import seaborn as sns
import random

%matplotlib inline
sns.set_style("whitegrid")
```

In [2]:

```
# dataframes creation for both training and testing datasets
fashion_train_df = pd.read_csv('/kaggle/input/fashionmnist/fashion-mnist_train.csv',sep=',')
fashion_test_df = pd.read_csv('/kaggle/input/fashionmnist/fashion-mnist_test.csv', sep = ',')
```

데이터 소개 및 전처리

In [3]:

```
# Let's view the head of the training dataset
# 784 indicates 28x28 pixels and 1 coloumn for the label
# After you check the tail, 60,000 training dataset are present
fashion_train_df.head()
```

Out[3]:

	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel776	pixel777	pixel778	pixe
0	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
1	9	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2	6	0	0	0	0	0	0	0	5	0	...	0	0	0	30	43
3	0	0	0	0	1	2	0	0	0	0	...	3	0	0	0	0
4	3	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0

5 rows × 785 columns

In [4]:

```
# Let's view the last elements in the training dataset
fashion_train_df.tail()
```

Out[4]:

	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel776	pixel777	pixel778
59995	9	0	0	0	0	0	0	0	0	0	...	0	0	0	0
59996	1	0	0	0	0	0	0	0	0	0	...	73	0	0	0
59997	8	0	0	0	0	0	0	0	0	0	...	160	162	163	135
59998	8	0	0	0	0	0	0	0	0	0	...	0	0	0	0
59999	7	0	0	0	0	0	0	0	0	0	...	0	0	0	0

5 rows × 785 columns

데이터 소개 및 전처리

In [5]:

```
# Let's view the head of the testing dataset
fashion_test_df.head()
```

Out[5]:

	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel776	pixel777	pixel778	pixe
0	0	0	0	0	0	0	0	0	9	8	...	103	87	56	0	0
1	1	0	0	0	0	0	0	0	0	0	...	34	0	0	0	0
2	2	0	0	0	0	0	0	14	53	99	...	0	0	0	0	63
3	2	0	0	0	0	0	0	0	0	0	...	137	126	140	0	133
4	3	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0

5 rows × 785 columns

In [6]:

```
# Let's view the last elements in the testing dataset
fashion_test_df.tail()
```

Out[6]:

	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel776	pixel777	pixel778
9995	0	0	0	0	0	0	0	0	0	0	...	32	23	14	20
9996	6	0	0	0	0	0	0	0	0	0	...	0	0	0	2
9997	8	0	0	0	0	0	0	0	0	0	...	175	172	172	182
9998	8	0	1	3	0	0	0	0	0	0	...	0	0	0	0
9999	1	0	0	0	0	0	0	0	140	119	...	111	95	75	44

5 rows × 785 columns

데이터 소개 및 전처리

In [7]:
```
fashion_train_df.shape
```

Out[7]:
```
(60000, 785)
```

In [8]:
```
# Create training and testing arrays
train = np.array(fashion_train_df, dtype='float32')
test = np.array(fashion_test_df, dtype='float32')
```

In [9]:
```
train.shape
```

Out[9]:
```
(60000, 785)
```

In [10]:
```
train
```

Out[10]:
```
array([[2., 0., 0., ..., 0., 0., 0.],
       [9., 0., 0., ..., 0., 0., 0.],
       [6., 0., 0., ..., 0., 0., 0.],
       ...,
       [8., 0., 0., ..., 0., 0., 0.],
       [8., 0., 0., ..., 0., 0., 0.],
       [7., 0., 0., ..., 0., 0., 0.]], dtype=float32)
```

In [11]:
```
test
```

Out[11]:
```
array([[0., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
       [2., 0., 0., ..., 0., 0., 0.],
       ...,
       [8., 0., 0., ..., 0., 1., 0.],
       [8., 0., 1., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.]], dtype=float32)
```

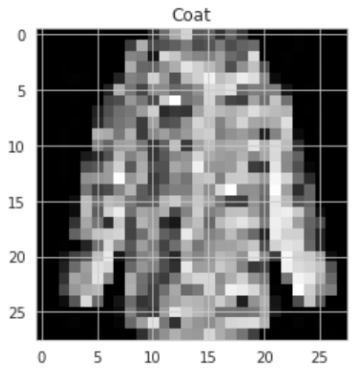
데이터 소개 및 전처리

In [12]:

```
class_names = ['T_shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Let's view some images!
i = random.randint(1,60000) # select any random index from 1 to 60,000
plt.imshow(train[i,1:].reshape((28,28))) # reshape and plot the image

plt.imshow(train[i,1:].reshape((28,28)) , cmap = 'gray') # reshape and plot the image
label_index = fashion_train_df["label"][i]
plt.title(f"{class_names[label_index]}")


# Remember the 10 classes decoding is as follows:
# 0 => T-shirt/top
# 1 => Trouser
# 2 => Pullover
# 3 => Dress
# 4 => Coat
# 5 => Sandal
# 6 => Shirt
# 7 => Sneaker
# 8 => Bag
# 9 => Ankle boot
```

Out[12]:

```
Text(0.5, 1.0, 'Coat')
```



In [13]:

```
label = train[i,0]
label
```

Out[13]:

```
4.0
```

데이터 소개 및 전처리

In [14]:

```
# Let's view more images in a grid format
# Define the dimensions of the plot grid
W_grid = 15
L_grid = 15

# fig, axes = plt.subplots(L_grid, W_grid)
# subplot return the figure object and axes object
# we can use the axes object to plot specific figures at various locations

fig, axes = plt.subplots(L_grid, W_grid, figsize=(17,17))

axes = axes.ravel() # flaten the 15 x 15 matrix into 225 array

n_train = len(train) # get the length of the train dataset

# Select a random number from 0 to n_train
for i in np.arange(0, W_grid * L_grid): # create evenly spaces variables

    # Select a random number
    index = np.random.randint(0, n_train)
    # read and display an image with the selected index
    axes[i].imshow( train[index,1:].reshape((28,28)) )
    label_index = int(train[index,0])
    axes[i].set_title(class_names[label_index], fontsize=8)
    axes[i].axis('off')

plt.subplots_adjust(hspace=0.4)

# Remember the 10 classes decoding is as follows:
# 0 => T-shirt/top
# 1 => Trouser
# 2 => Pullover
# 3 => Dress
# 4 => Coat
# 5 => Sandal
# 6 => Shirt
# 7 => Sneaker
# 8 => Bag
# 9 => Ankle boot
```

데이터 소개 및 전처리

In [15]:

```
# Prepare the training and testing dataset
X_train = train[:, 1:] / 255
y_train = train[:, 0]

X_test = test[:, 1:] / 255
y_test = test[:,0]
```

In [16]:

```
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i].reshape((28,28)), cmap=plt.cm.binary)
    label_index = int(y_train[i])
    plt.title(class_names[label_index])
plt.show()
plt.tight_layout()
```

```
<Figure size 432x288 with 0 Axes>
```

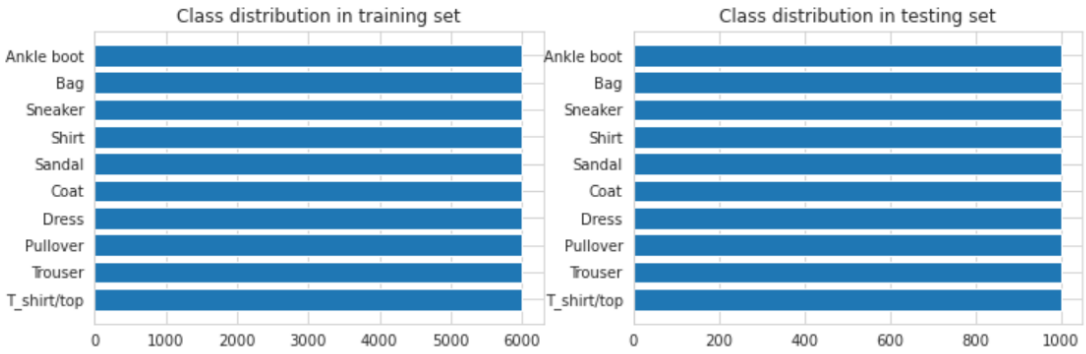
데이터 소개 및 전처리

In [17]:

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
classes, counts = np.unique(y_train, return_counts=True)
plt.barh(class_names, counts)
plt.title('Class distribution in training set')

plt.subplot(2, 2, 2)
classes, counts = np.unique(y_test, return_counts=True)
plt.barh(class_names, counts)
plt.title('Class distribution in testing set')
```

Out[17]:

```
Text(0.5, 1.0, 'Class distribution in testing set')
```



- 훈련 데이터와 테스트 데이터 모두 클래스들이 비슷한 분포를 가지고 있음을 시각화를 통해 확인
- 모델이 모든 클래스에 대해 균형 잡힌 학습을 할 수 있다는 점을 알 수 있음

모델 구성

In [18]:
```
from sklearn.model_selection import train_test_split

X_train, X_validate, y_train, y_validate = train_test_split(X_train, y_train, test_size=0.2, rand
om_state=42)
```

In [19]:
```
print(X_train.shape)
print(y_train.shape)
```

```
(48000, 784)
```

In [20]:
```
# * unpack the tuple
X_train = X_train.reshape(X_train.shape[0], * (28, 28, 1))
X_test = X_test.reshape(X_test.shape[0], * (28, 28, 1))
X_validate = X_validate.reshape(X_validate.shape[0], * (28, 28, 1))
```

In [21]:
```
print(X_train.shape)
print(y_train.shape)
print(X_validate.shape)
print(y_validate.shape)
```

```
(48000, 28, 28, 1)
(48000,)
(12000, 28, 28, 1)
(12000,)
```

모델 구성

In [22]:

```
import keras
import tensorflow as tf
```

In [23]:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout, BatchNormaliza
tion
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import TensorBoard
```

모델 구성

```
cnn_model = Sequential()
```

```
cnn_model.add(Conv2D(filters=32, kernel_size=(3, 3), input_shape=(28,28,1), activation='relu', pa
dding='same'))
cnn_model.add(BatchNormalization())
cnn_model.add(Conv2D(filters=32, kernel_size=(3, 3), input_shape=(28,28,1), activation='relu', pa
dding='same'))
cnn_model.add(BatchNormalization())
```

- Keras의 Sequential() 모델을 사용해 순차적인 레이어 쌓기 방식으로 CNN 구축
- 두 개의 Conv2D 레이어로 특징 추출
- 활성화함수로 Relu 함수 사용
- Padding = 'same'으로 출력 이미지 크기를 입력과 같게 유지
- BatchNormalization()을 통해 배치 정규화로 학습을 더 빠르고 안정적으로 만듦

모델 구성

```
cnn_model.add(MaxPooling2D(pool_size=(2, 2)))
cnn_model.add(Dropout(0.2))
```

- 2X2 크기의 필터로 풀링을 이용하여 특징 맵 크기를 줄임
- 과적합(overfitting) 문제를 막기 위해 드롭아웃 기법을 이용해 20%의 뉴런을 무작위로 제거

모델 구성

```
cnn_model.add(Conv2D(filters=64, kernel_size=(3, 3), input_shape=(28,28,1), activation='relu', pa
dding='same'))
cnn_model.add(BatchNormalization())
cnn_model.add(Conv2D(filters=64, kernel_size=(3, 3), input_shape=(28,28,1), activation='relu', pa
dding='same'))
cnn_model.add(BatchNormalization())
cnn_model.add(MaxPooling2D(pool_size=(2, 2)))
cnn_model.add(Dropout(0.2))
```

- 앞의 슬라이드와 비슷하지만 필터의 개수를 64개로 증가해 더 많은 특징을 추출
- 풀링과 드롭아웃으로 추출한 내용들을 정리

모델 컴파일 및 학습

```
cnn_model.add(Flatten())
cnn_model.add(Dense(units=128, activation='relu'))
cnn_model.add(Dropout(0.2))
cnn_model.add(Dense(units=10, activation='softmax'))
```

- CNN으로 추출한 2D를 1D 벡터로 변환하여 Dense 층에 전달
- Relu 함수를 활성화 함수로 하여 128개의 뉴런을 가진 은닉층 생성
- 20%의 뉴런을 드롭아웃
- Softmax 함수를 이용해 각 클래스에 대한 확률 분포 출력

모델 컴파일 및 학습

In [25]:

```
METRICS = [
    'accuracy',
    tf.keras.metrics.Precision(name='precision'),
    tf.keras.metrics.Recall(name='recall')
]

cnn_model.compile(loss ='sparse_categorical_crossentropy', optimizer='adam' ,metrics=['accurac
y'])
```

- 정답이 정수일 때 사용하는 다중 분류 손실 함수인 sparse_categorical_crossentropy 함수를 사용
- Optimizer는 adam optimizer를 사용해 빠르고 안정적인 최적화 알고리즘을 사용
- 정확도(accuracy)를 사용해 정답과 예측이 어느 정도로 일치하는지 측정

모델 컴파일 및 학습

In [26]:

```
epochs = 50
batch_size = 512

history = cnn_model.fit(
    X_train, y_train,
    batch_size=batch_size,
    epochs=epochs,
    verbose=1,
    validation_data=(X_validate, y_validate)
)
```

- epochs를 50, batch_size를 512로 학습
- X_train, y_train을 학습용 입력 데이터와 레이블에 넣고, validation_data를 이용해 학습 중간에 검증용 데이터를 사용해 모델 평가

```
Epoch 1/50
94/94 [==============================] - 10s 25ms/step - loss: 0.6645 - accuracy: 0.7707 - val
_loss: 2.4232 - val_accuracy: 0.2312
Epoch 2/50
94/94 [==============================] - 2s 21ms/step - loss: 0.3642 - accuracy: 0.8661 - val_
loss: 2.8972 - val_accuracy: 0.1543
Epoch 3/50
94/94 [==============================] - 2s 22ms/step - loss: 0.3015 - accuracy: 0.8888 - val_
loss: 3.0699 - val_accuracy: 0.2123
Epoch 4/50
94/94 [==============================] - 2s 21ms/step - loss: 0.2696 - accuracy: 0.9014 - val_
loss: 2.1257 - val_accuracy: 0.4398
Epoch 5/50
94/94 [==============================] - 2s 21ms/step - loss: 0.2481 - accuracy: 0.9075 - val_
loss: 1.0407 - val_accuracy: 0.6679
Epoch 6/50
94/94 [==============================] - 2s 21ms/step - loss: 0.2268 - accuracy: 0.9168 - val_
loss: 0.5070 - val_accuracy: 0.8270
Epoch 7/50
94/94 [==============================] - 2s 21ms/step - loss: 0.2127 - accuracy: 0.9219 - val_
loss: 0.2566 - val_accuracy: 0.9101
Epoch 8/50
94/94 [==============================] - 2s 22ms/step - loss: 0.1996 - accuracy: 0.9275 - val_
loss: 0.2527 - val_accuracy: 0.9084
Epoch 9/50
94/94 [==============================] - 2s 21ms/step - loss: 0.1857 - accuracy: 0.9309 - val_
loss: 0.2062 - val_accuracy: 0.9267
Epoch 10/50
94/94 [==============================] - 2s 21ms/step - loss: 0.1797 - accuracy: 0.9342 - val_
loss: 0.2142 - val_accuracy: 0.9250
```

```
Epoch 46/50
94/94 [==============================] - 2s 21ms/step - loss: 0.0418 - accuracy: 0.9850 - val_
loss: 0.2977 - val_accuracy: 0.9374
Epoch 47/50
94/94 [==============================] - 2s 21ms/step - loss: 0.0428 - accuracy: 0.9842 - val_
loss: 0.2841 - val_accuracy: 0.9375
Epoch 48/50
94/94 [==============================] - 2s 22ms/step - loss: 0.0430 - accuracy: 0.9845 - val_
loss: 0.3036 - val_accuracy: 0.9382
Epoch 49/50
94/94 [==============================] - 2s 21ms/step - loss: 0.0423 - accuracy: 0.9839 - val_
loss: 0.2826 - val_accuracy: 0.9365
Epoch 50/50
94/94 [==============================] - 2s 21ms/step - loss: 0.0413 - accuracy: 0.9848 - val_
loss: 0.3016 - val_accuracy: 0.9388
```

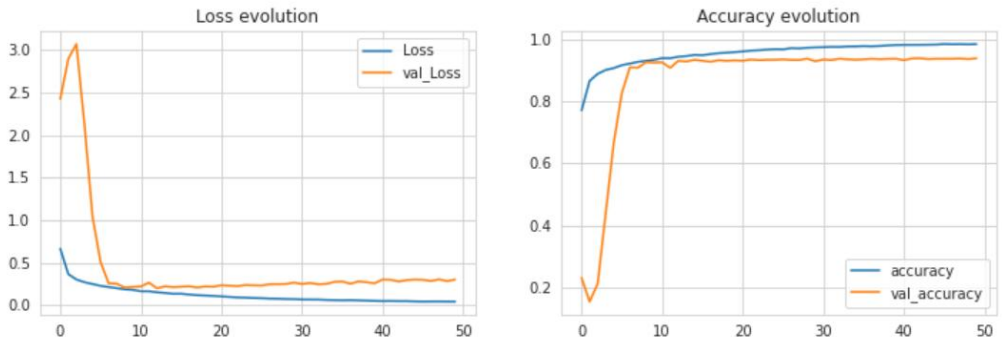
모델 컴파일 및 학습

In [27]:

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['val_loss'], label='val_Loss')
plt.legend()
plt.title('Loss evolution')

plt.subplot(2, 2, 2)
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.legend()
plt.title('Accuracy evolution')
```

Out[27]:

```
Text(0.5, 1.0, 'Accuracy evolution')
```



- 왼쪽 그래프는 손실(loss) 변화에 대한 그래프
- 오른쪽 그래프는 정확도(accuracy) 변화에 대한 그래프

학습 결과 및 예측

In [28]:

```
evaluation = cnn_model.evaluate(X_test, y_test)
print(f'Test Accuracy : {evaluation[1]:.3f}')
```

```
313/313 [==============================] - 1s 3ms/step - loss: 0.2755 - accuracy: 0.9369
Test Accuracy : 0.937
```

- 테스트 데이터셋에서 정확도(accuracy)가 93.7%를 기록
- 훈련 데이터보다 약간 낮은 정확도인데 이는 과적합(overfitting)이 나타났음을 알 수 있음
- 이를 해결하기 위해 드롭아웃 비율을 20%에서 30%정도로 높이거나 epochs 수를 줄이는 등의 방법으로 해결 가능

학습 결과 및 예측 - 개선

```
evaluation = cnn_model.evaluate(X_test, y_test)
print(f'Test Accuracy : {evaluation[1]:.3f}')
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9318 - loss: 0.2051
Test Accuracy : 0.935
```

- Epochs 수를 50에서 10으로 줄여서 학습
- 정확도는 93.5%로 비슷하지만 손실(loss)은 0.2755에서 0.2051로 줄어 덜 과적합된 것을 알 수 있음

```
evaluation = cnn_model.evaluate(X_test, y_test)
print(f'Test Accuracy : {evaluation[1]:.3f}')
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - accuracy: 0.9207 - loss: 0.2130
Test Accuracy : 0.921
```

- Epochs 수를 50에서 10, dropout을 20%에서 30%으로 줄여서 학습
- 손실(loss)이 0.2755에서 0.2130으로 줄어 과적합이 덜 된 것을 알 수 있음

학습 결과 및 예측

```
predicted_classes = cnn_model.predict(X_test)
predicted_classes = np.argmax(predicted_classes, axis=1)
```

In [30]:

```
test_img = X_test[0]
prediction = cnn_model.predict(X_test)
prediction[0]
```

Out[30]:

```
array([1.0000000e+00, 6.0717133e-18, 8.5047762e-13, 6.2204925e-10,
       2.3696255e-15, 1.9046340e-17, 4.5268173e-08, 1.4423075e-17,
       1.5776774e-11, 5.5990565e-17], dtype=float32)
```

In [31]:

```
np.argmax(prediction[0])
```

Out[31]:

```
0
```

- 테스트 데이터 중에서 predicted model을 선택
- 결과를 통해 predicted model이 클래스 0번(T-shirt/top)인 것을 알 수 있음

학습 결과 및 예측

In [32]:

```
L = 5
W = 5
fig, axes = plt.subplots(L, W, figsize = (12,12))
axes = axes.ravel()

for i in np.arange(0, L * W):
    axes[i].imshow(X_test[i].reshape(28,28))
    axes[i].set_title(f"Prediction Class = {(predicted_classes[i]):0.1f}\n True Class = {y_test
[i]:0.1f}")
    axes[i].axis('off')

plt.subplots_adjust(wspace=0.5)
```

- 그림을 통해 True Class의 번호와 Predicted Class의 번호가 같음을 알 수 있음

학습 결과 및 예측

In [34]:

```
from sklearn.metrics import classification_report

num_classes = 10
target_names = [f"Class {i}" for i in range(num_classes)]

print(classification_report(y_test, predicted_classes, target_names = target_names))
```

```
              precision    recall  f1-score   support

     Class 0       0.86      0.90      0.88      1000
     Class 1       0.99      0.99      0.99      1000
     Class 2       0.91      0.91      0.91      1000
     Class 3       0.94      0.95      0.94      1000
     Class 4       0.91      0.93      0.92      1000
     Class 5       1.00      0.98      0.99      1000
     Class 6       0.84      0.77      0.81      1000
     Class 7       0.96      0.98      0.97      1000
     Class 8       0.99      1.00      0.99      1000
     Class 9       0.97      0.97      0.97      1000

    accuracy                           0.94     10000
   macro avg       0.94      0.94      0.94     10000
weighted avg       0.94      0.94      0.94     10000
```

- 정밀도(precision), 재현율(recall), f1-score(precision,과 recall의 조화 평균)을 토대로 각 클래스에 대한 개별 지표 분석
- 전체적으로 정확도는 94%로 학습이 잘 되었다는 것을 알 수 있고, 특히 Class 1, 5, 8에서 거의 완벽하게 학습이 이루어짐
- 다만, Class 6의 정확도가 84%로 다른 클래스에 비해 다소 낮은 성능을 보였는데 이는 Class 6이 shirt임으로 유사한 이미지 특성 때문일 것으로 예상됨

결론

- CNN을 활용해 Fashion MNIST 이미지 분류를 성공적으로 함
- 정확도 94%로 높은 성능으로 학습함
- Relu 함수, Softmax 함수, Adam Optimizer, Dropout, 배치 정규화, 풀링 등의 다양한 기법들로 모델을 구성
- 약간의 과적합이 일어났지만 드롭아웃 비율과 에폭 수 조절로 해결
- 클래스별 정확도 차이 분류를 통해 일부 클래스의 성능이 낮음을 확인 및 이유 분석